

Функциональное программирование

Haskell

Haskell – чистый функциональный язык программирования со строгой статической типизацией.

О Haskell в контексте типизации:

Статическая типизация: типы всех выражений проверяются на этапе компиляции, а не во время выполнения.

Строгость: язык не допускает неявных преобразований и «тихих» ошибок из-за несоответствия типов.

Вывод типов: во многих случаях тип вычисляется автоматически (на основе структуры выражения и контекста), поэтому явное указание типа не обязательно, но возможно.

Haskell — чистый функциональный язык программирования со строгой статической типизацией.

Что значит «функциональный»:

В функциональной парадигме программа — это композиция функций, которые преобразуют входные данные в выходные. Это ближе к математическому понятию функции: $f(x) = y$, а не к последовательности команд «сделай А, потом В, потом измени С».

Ключевые черты:

Декларативность: вы описываете *что* должно получиться, а не *как* пошагово достичь результата.

Функции как значения: их можно передавать как аргументы, возвращать из других функций, хранить в структурах данных (функции высшего порядка).

Рекурсия вместо циклов: повторяющиеся вычисления выражаются через рекурсивные вызовы, а не через изменяемые счётчики и циклы.

Haskell — чистый функциональный язык программирования со строгой статической типизацией.

Что значит «чистый»:

«Чистота» относится к функциям: чистая функция удовлетворяет двум условиям:

Детерминированность: для одних и тех же входных аргументов она всегда возвращает один и тот же результат.

Отсутствие побочных эффектов: она не меняет внешнее состояние и не взаимодействует с внешним миром «по пути».

Что считается побочным эффектом:

Примеры побочных эффектов, которых избегают в чистых функциях:
изменение глобальной переменной или мутация структуры данных;
ввод/вывод (чтение файла, вывод в консоль, запрос по сети);
работа с текущим временем, генерация случайных чисел;
выброс исключений как основной способ управления потоком.

Data.Bool

Bool — логический тип

Значения: True и False.

Назначение: условия, флаги, результаты проверок.

Основные операции:

С Bool работают стандартные логические операторы:

&& — логическое «И» (and)

|| — логическое «ИЛИ» (or)

not — логическое отрицание (not)

Otherwise — псевдоним для True

Data.Char

Char — отдельный символ

Тип для одного символа, пишется в одинарных кавычках, например 'a', '1', '\n'.

Кодировка: современный Haskell работает с Unicode, так что '😊' — валидный Char.

String — это не отдельный «особый» тип, а **псевдоним для списка символов**: Поэтому все операции над списками работают и со строками.

Как создаются и выглядят строки:

Строковые литералы пишутся в **двойных кавычках**: "hello".

Это то же самое, что список Char: "abc" $\equiv$ ['a','b','c'].

Пустая строка — "" $\equiv$ [].

Экранирование: "He said \"Hi\"" $\rightarrow$ He said "Hi".

Предикаты: проверки принадлежности символа к классу

ВСЕ ПРЕДИКАТЫ ИМЕЮТ ТИП CHAR -> BOOL.

Принадлежность к диапазонам кодировки:

Функция	Что проверяет
isAscii	Символ входит в первые 128 символов Unicode (ASCII)
isLatin1	Символ входит в первые 256 символов Unicode (Latin-1)
isAsciiUpper	Символ — заглавная ASCII-буква ('A'-'Z')
isAsciiLower	Символ — строчная ASCII-буква ('a'-'z')

Цифры и числа:

Функция	Что проверяет
isDigit	ASCII-цифра ('0'-'9')
isOctDigit	Восьмеричная цифра ('0'-'7')
isHexDigit	Шестнадцатеричная цифра ('0'-'9', 'a'-'f', 'A'-'F')
isNumber	Числовой символ Unicode (включает римские цифры, цифры других письменностей и т. д.)

Предикаты: проверки принадлежности символа к классу

Буквы и регистр:

Функция	Что проверяет
isAlpha	Буква любого регистра (включая буквы без регистра из других письменностей)
isLetter	То же, что isAlpha (синоним)
isLower	Строчная буква (Unicode)
isUpper	Заглавная или title-case буква (Unicode)
isAlphaNum	Буква или цифра (Unicode)

Пробелы и разделители:

Функция	Что проверяет
isSpace	Пробельный символ Unicode, а также \t, \n, \r, \f, \v
isSeparator	Unicode-разделитель (Space, LineSeparator, ParagraphSeparator)

Знаки и символы:

Функция	Что проверяет
isPunctuation	Знак препинания (включая скобки, кавычки, соединители)
isSymbol	Символ (математический, валютный, модификаторный и т. д.)
isMark	Unicode-знак-модификатор (диакритика, комбинирующиеся символы)

Печатность и управление:

Функция	Что проверяет
isControl	Управляющий символ
isPrint	Печатный символ (буквы, цифры, знаки, пробелы — всё, что не control)

Преобразование регистра, конвертация цифр

Преобразование регистра:

Функция	Тип	Что делает
toUpper	Char -> Char	Преобразует букву в заглавную
toLowerCase	Char -> Char	Преобразует букву в строчную
toTitle	Char -> Char	Преобразует букву в title-case (отличается от upper для небольшого числа лигатур)

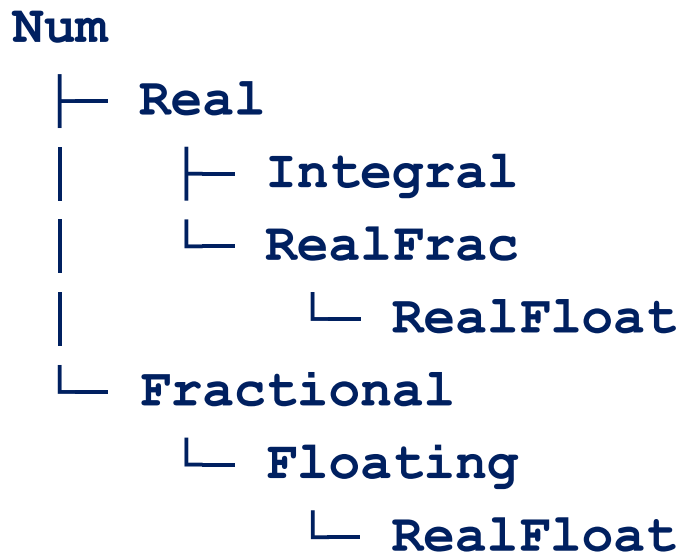
Кодирование: символ ↔ код:

Функция	Тип	Что делает
ord	Char -> Int	Возвращает Unicode-код символа (ord 'a' → 97)
chr	Int -> Char	Возвращает символ по Unicode-коду (chr 97 → 'a')

Конвертация цифр:

Функция	Тип	Что делает
digitToInt	Char -> Int	Превращает символ-цифру в число. Поддерживает шестнадцатеричные цифры: 'a' → 10, 'F' → 15. Завершается ошибкой, если символ не является hex-цифрой
intToDigit	Int -> Char	Обратное преобразование: 0 → '0', 10 → 'a', 15 → 'f'. Диапазон 0–15, иначе ошибка

Иерархия числовых классов типов



На практике чаще всего используют такие цепочки:

Целые числа: Num → Real → Integral (сюда входят Int, Integer, Word и т. п.)

Дробные числа: Num → Fractional → Floating (сюда входят Float, Double, Complex а при условии, что a – Floating)

Важное уточнение:

RealFloat — это «общий наследник» для тех типов, которые одновременно и дробные, и вещественные (поддерживают и деление, и округление, и тригонометрию).

Классы типов задают операции:

Num — базовые арифметические операции.

Integral — целочисленное деление и остаток (div, mod, quot, rem).

Fractional — обычное деление (/).

Floating — тригонометрия, экспоненты, логарифмы.

RealFloat — низкоуровневые операции с плавающей точкой (мантисса, экспонента, проверка на NaN, бесконечность).

Num (GHC.Num)

Num — это базовый класс типов для чисел в Haskell. Он задаёт минимальный набор операций, которые «должны быть у любого числа», и именно благодаря ему работают числовые литералы вроде 42 или 3.14.

Литерал без точки (например, 42) имеет тип `Num a => a`. Компилятор подставляет конкретный тип, когда видит контекст.

```
class Num a where
    (+)      :: a -> a -> a
    (-)      :: a -> a -> a
    (*)      :: a -> a -> a
    negate   :: a -> a
    abs      :: a -> a
    signum    :: a -> a
    fromInteger :: Integer -> a
```

Ключевые моменты:

fromInteger — критически важная функция: она объясняет, почему литерал 42 можно использовать как `Int`, `Integer`, `Double` и т. д. Компилятор берёт целое `Integer` и превращает его в нужный тип через `fromInteger`.

signum возвращает «знак»: -1, 0 или 1 (для комплексных чисел смысл немного другой).

abs — модуль (абсолютное значение).

Eq (Data.Eq)

Eq — это базовый класс типов для **сравнения на равенство**. Он даёт возможность проверять, «одинаковы ли два значения», и нужен почти везде: от простых проверок до поиска в списках и использования в структурах данных (например, в множествах или словарях).

```
class Eq a where
    (==) :: a -> a -> Bool
    (/=) :: a -> a -> Bool
```

При этом по умолчанию один метод выражается через другой. Поэтому достаточно реализовать **только один** из них, второй появится автоматически.

Eq (Data.Eq), Show (GHC.Show)

Show — это класс типов, который отвечает за **преобразование значения в строку** для вывода (в консоль, лог, отладку).

Его главная функция: `show :: a -> String`.

```
class Show a where
  showsPrec :: Int -> a -> ShowS
  show      :: a -> String
  showList :: [a] -> ShowS
```

```
data Color = Red | Green | Blue
```

```
instance Show Color where
  show Red    = "Red"
  show Green  = "Green"
  show Blue   = "Blue"
```

```
instance Eq Color where
  Red == Red    = True
  Green == Green = True
  Blue == Blue  = True
  _ == _        = False
```

Ord (Data.Ord)

Ord — это класс типов для **упорядоченных значений**, у которых есть осмысленное понятие «меньше/больше». Он критически важен для сортировки, поиска, бинарных деревьев и вообще любых алгоритмов, где нужно сравнивать элементы.

```
class Eq a => Ord a where
    compare :: a -> a -> Ordering
    (<)      :: a -> a -> Bool
    (<=)     :: a -> a -> Bool
    (>)      :: a -> a -> Bool
    (>=)     :: a -> a -> Bool
    max      :: a -> a -> a
    min      :: a -> a -> a
```

где data Ordering = LT | EQ | GT.

Ключевые моменты:

compare $x\ y$ — минимально необходимая функция (всё остальное можно реализовать через неё).

Все остальные операторы и $\max/\min$ имеют реализации по умолчанию.

Обязательное условие:

тип должен быть экземпляром Eq (равенство нужно, чтобы корректно определять EQ).

Порядок должен быть согласован с равенством:

если $x == y$, то $\text{compare } x\ y == EQ$. Иначе алгоритмы (вроде бинарного поиска) могут работать некорректно.

```
data Priority = Low | Medium | High
```

```
instance Ord Priority where
    compare Low Medium = LT
    compare Medium High = LT
    compare Low High   = LT
    compare x y
        | x == y      = EQ
        | otherwise   = GT
```

Real (GHC.Real)

Real — это класс типов, который описывает **вещественно упорядочиваемые числа**, которые можно превратить в рациональное представление.

```
class (Num a, Ord a) => Real a where
    toRational :: a -> Rational
```

То есть:

Тип должен быть экземпляром Num и Ord (чтобы можно было сравнивать).

Единственная обязательная функция — toRational, которая превращает значение типа a в точное рациональное число.

Зачем нужен toRational:

Это способ получить **точное представление** числа без потерь, свойственных плавающей точке. Например:

```
toRational (0.1 :: Double)
```

```
-- Результат: 3602879701896397 % 36028797018963968
```

```
-- Это точное значение, которое реально хранится в Double для 0.1
```

Для целых чисел всё проще:

```
toRational (5 :: Int)    -- 5 % 1
```

```
toRational (100 :: Integer) -- 100 % 1
```

Integral (GHC.Real)

Integral — это класс типов для **целых чисел**, который добавляет к уже знакомым Num и Real операции **целочисленного деления и взятия остатка**, а также преобразование в Integer.

Он нужен, когда важно работать именно с целыми значениями и получать целые результаты (без дробной части).

```
class (Real a, Enum a) => Integral a where
    quot, rem          :: a -> a -> a
    div, mod           :: a -> a -> a
    quotRem, divMod    :: a -> a -> (a, a)
    toInteger          :: a -> Integer
```

Основные экземпляры Integral :

Int — целые числа фиксированной ширины (обычно 32 или 64 бита), могут переполняться.

Integer — целые числа произвольной точности, переполнения нет.

Integral (GHC.Real)

quot / rem — «усекающее» деление и остаток (знак остатка совпадает со знаком делимого).

div / mod — «пол»-деление и остаток (результат округляется вниз к $-\infty$, остаток имеет тот же знак, что и делитель).

quotRem и **divMod** возвращают пару (частное, остаток) за один проход — это эффективнее, чем вызывать **div** и **mod** по отдельности.

toInteger позволяет превратить любое Integral-значение в «универсальный» Integer (произвольной точности).

Разница между div/mod и quot/rem:

Для положительных чисел разницы нет, но для отрицательных она проявляется:

```
(-7) `div` 3    -- -3 (округление вниз)
```

```
(-7) `mod` 3    -- 2 (остаток >= 0 при положительном делителе)
```

```
(-7) `quot` 3   -- -2 (усечение к нулю)
```

```
(-7) `rem` 3    -- -1 (знак совпадает с делимым)
```

Практическое правило:

в алгоритмах почти всегда берут **div** и **mod**, потому что они ведут себя предсказуемо с отрицательными числами и согласованы с математическим определением деления с остатком.

RealFrac (Numeric.RealFrac)

RealFrac — это класс типов для **вещественных дробных чисел**, у которых можно выделить целую и дробную части и выполнять разные виды округления.

Он объединяет свойства `Real` (упорядоченность, возможность превращения в рациональное) и `Fractional` (деление), и добавляет операции, специфичные для работы с дробными значениями.

```
class (Real a, Fractional a) => RealFrac a where
    properFraction :: (Integral b) => a -> (b, a)
    truncate      :: (Integral b) => a -> b
    round         :: (Integral b) => a -> b
    floor         :: (Integral b) => a -> b
    ceiling       :: (Integral b) => a -> b
```

Основные экземпляры **RealFrac** :

Float, Double — стандартные типы с плавающей точкой.

Rational — рациональные числа.

Ключевые функции:

properFraction `x` — возвращает пару (целая_часть, дробная_часть).

Целая часть имеет тип `Integral b => b`, дробная — того же типа, что и `x`.

truncate `x` — отбрасывает дробную часть (округление к нулю).

round `x` — математическое округление (к ближайшему целому; при «ровно посередине» — к чётному в GHC).

floor `x` — округление вниз (к $-\infty$).

ceiling `x` — округление вверх (к $+\infty$).

Все эти функции используют `properFraction` как базовую операцию: остальные можно реализовать через неё.

Fractional (Numeric.Fractional)

Fractional - это класс типов для числовых типов, поддерживающих **вещественное деление**. Он расширяет Num, добавляя операции, необходимые для работы с дробными значениями.

```
class Num a => Fractional a where
    (/)      :: a -> a -> a
    recip    :: a -> a
    fromRational :: Rational -> a
```

Основные экземпляры Fractional :

Float, Double — числа с плавающей точкой.

Rational — рациональные числа (точное представление в виде дроби).

Complex *a* (при условии, что *a* — RealFloat) — комплексные числа.

Типы вроде Int и Integer **не являются** экземплярами Fractional, потому что деление целых чисел не всегда даёт целое число. Для них используют Integral и операции div/mod.

Ключевые функции:

(/) — вещественное деление.

В отличие от div (целочисленное деление), / возвращает дробный результат.

Пример: $5 / 2 \rightarrow 2.5$ (для Double).

recip *x* — обратное значение, то есть $1 / x$. Удобно, когда нужно явно выразить «обратную величину».

fromRational — преобразует рациональное число в значение типа *a*.

Это важно для литералов:

когда вы пишете 3.14, Haskell на самом деле использует fromRational (314 % 100).

Floating (Numeric. Floating)

Floating — это класс типов для чисел, на которых можно выполнять трансцендентные операции:

тригонометрию, экспоненты, логарифмы и т. п.

Он расширяет Fractional, поэтому все экземпляры Floating обязаны поддерживать и обычное деление.

Основные экземпляры Floating :

Float, Double — числа с плавающей точкой.

Complex a (при условии, что a — RealFloat) — комплексные числа.

Возведение в степень ^, ^^ и **:

^ — возведение в целую неотрицательную степень (Num).

^^ — возведение в произвольную целую степень (Fractional).

****** — возведение в произвольную вещественную степень (Floating).

Выбор оператора зависит от требуемой точности и типа показателя.

```
class Fractional a => Floating a where
    pi      :: a
    exp     :: a -> a
    log     :: a -> a
    sqrt    :: a -> a
    (**)    :: a -> a -> a
    logBase :: a -> a -> a
    sin     :: a -> a
    cos     :: a -> a
    tan     :: a -> a
    asin    :: a -> a
    acos    :: a -> a
    atan    :: a -> a
    sinh    :: a -> a
    cosh    :: a -> a
    tanh    :: a -> a
    asinh   :: a -> a
    acosh   :: a -> a
    atanh   :: a -> a
```

RealFloat (Numeric.RealFloat)

RealFloat — это специализированный класс типов для чисел с плавающей точкой, который объединяет возможности **RealFrac** и **Floating** и добавляет низкоуровневые операции для работы с внутренним представлением чисел, а также проверки на особые состояния вроде NaN или бесконечности.

Ключевые группы функций:

Параметры типа и представление

- `floatRadix` — основание системы счисления (обычно 2).
- `floatDigits` — количество значащих цифр в мантиссе.
- `floatRange` — диапазон допустимых значений экспоненты.

Разбор и сборка числа

- `decodeFloat x` возвращает пару (m, e) такую, что $x = m \cdot b^e$.
- `encodeFloat m e` делает обратное: собирает число из мантиссы и экспоненты.

Компоненты числа в более удобном виде

- `exponent x` — экспонента числа x.
- `significand x` — нормализованная мантисса (в интервале (-1,1)).
- `scaleFloat k x` — умножает число на b^k (быстрое масштабирование экспоненты).

Проверки на особые случаи

- `isNaN`, `isInfinite`, `isDenormalized`, `isNegativeZero` — проверки на специальные значения.
- `isIEEE` — сообщает, следует ли тип стандарту IEEE.

Дополнительные математические операции

- `atan2 y x` — арктангенс с учётом квадранта (угол вектора (x,y)).

```
class (RealFrac a, Floating a) => RealFloat a where
    floatRadix      :: a -> Integer
    floatDigits     :: a -> Int
    floatRange      :: a -> (Int, Int)
    decodeFloat     :: a -> (Integer, Int)
    encodeFloat     :: Integer -> Int -> a
    exponent        :: a -> Int
    significand     :: a -> a
    scaleFloat      :: Int -> a -> a
    isNaN           :: a -> Bool
    isInfinite      :: a -> Bool
    isDenormalized  :: a -> Bool
    isNegativeZero  :: a -> Bool
    isIEEE          :: a -> Bool
    atan2           :: a -> a -> a
```

Основные экземпляры RealFloat :

Float — числа с плавающей точкой одинарной точности.

Double — числа с плавающей точкой двойной точности.

Float и Double в Haskell

Базовые характеристики:

Параметр	Float	Double
Размер	32 бита (обычно)	64 бита (обычно)
Формат	IEEE 754 binary32	IEEE 754 binary64
Точность	~6–7 десятичных знаков	~15–17 десятичных знаков
Диапазон	$\sim 1,4 \cdot 10^{-45}$ до $3,4 \cdot 10^{38}$	$\sim 4,9 \cdot 10^{-324}$ до $1,8 \cdot 10^{308}$
Память	Меньше	Больше
Скорость (в ряде сценариев)	Часто быстрее (SIMD, GPU)	Чуть медленнее при массовых операциях

Оба типа реализуют одни и те же классы:

Num, Fractional, Floating, RealFrac, RealFloat.

Преобразования между Float и Double:

В Haskell нет неявных преобразований, используются специальные функции из GHC.Float:

```
import GHC.Float (float2Double, double2Float)

f :: Float
f = 3.14

d :: Double
d = float2Double f      -- Float -> Double

f2 :: Float
f2 = double2Float d     -- Double -> Float (возможна потеря точности)
```

Float и Double в Haskell

Выбирают Float, если:

- Нужно экономить память в больших массивах (например, миллионы координат, текстуры, сырые данные).
- Целевая платформа (GPU, некоторые SIMD-инструкции) заметно быстрее работает с одинарной точностью.
- Формат данных (файлы, сетевые протоколы, графика) изначально рассчитан на 32-битные числа.
- Точность 6–7 знаков достаточна для задачи (например, визуализация, игровые координаты, фильтрация сигналов без жёстких требований).

Выбирают Double, если:

- Важна высокая точность: научные расчёты, моделирование, численные методы.
- Длинные цепочки вычислений, где ошибки округления накапливаются.
- Задача чувствительна к малым различиям (например, сравнение близких значений, вычисление производных).
- Стандартные библиотеки и внешние зависимости по умолчанию используют Double.

Не подходят ни для чего финансового: ни Float, ни Double не годятся для денег из-за ошибок округления. Для этого используют фиксированную точку, Rational или специализированные типы.

Особые значения NaN, +0, -0, Infinity, -Infinity

```
import GHC.Float (isNaN, isInfinite, isNegativeZero)
```

```
isInfinite (1/0)      -- True
isInfinite (-1/0)     -- True
```

```
0.0 == (-0.0)         -- True (по значению равны)
isNegativeZero 0.0     -- False
isNegativeZero (-0.0)  -- True
```

NaN (Not-a-Number) — это специальное значение чисел с плавающей точкой по стандарту IEEE 754.

В Haskell оно встречается в типах Float и Double. Учитывая твои учебные практики (алгоритмы, структуры данных, обработка данных), важно понимать не просто «что это», а **как это ломает обычные операции и как с этим безопасно работать.**

```
nan :: Double
nan = 0/0
```

```
nan2 :: Double
nan2 = sqrt (-1)
```

Оба выражения дают NaN.

```
nan == nan  -- False!
```

```
nan /= nan  -- True
```

Это самое важное: **никакое сравнение через == не поможет обнаружить NaN.**

- Поскольку NaN /= NaN, поведение Eq и Ord становится «необычным»:
- NaN == x всегда False для любого x (включая сам NaN).
- Любой оператор порядка (<, <=, >, >=) с участием NaN возвращает False.
- compare NaN NaN вернёт GT (по реализации в GHC), но это не «нормальный» порядок: он нарушает интуитивные ожидания.

Следствие: сортировка и структуры данных, опирающиеся на транзитивность и рефлексивность, могут вести себя странно, если в данных есть NaN.

```
import GHC.Float (isNaN)
```

```
isNaN nan      -- True
```

```
isNaN 3.14     -- False
```

```
isNaN (1/0)    -- False (это Infinity, не NaN)
```

Приведение числовых типов данных

Откуда → куда	Функция / способ	Нюанс	Пример
Int → Double	fromIntegral	Безопасно, но явно	fromIntegral (5 :: Int) :: Double
Int → Float	fromIntegral	Возможна потеря точности при очень больших Int	fromIntegral (10 :: Int) :: Float
Double → Int	round / floor / truncate (+ fromIntegral)	Нужно выбрать стратегию округления	truncate (3.7 :: Double) :: Int
Float → Int	round / floor / ceiling / truncate (+ fromIntegral)	Выбор функции зависит от логики округления; результат — Integer, нужен дополнительный fromIntegral для Int	fromIntegral (round (2.3 :: Float)) :: Int
Float → Double	float2Double	Без потерь внутри формата	float2Double (1.5 :: Float) :: Double
Double → Float	double2Float	Возможна потеря точности	double2Float (3.14159 :: Double) :: Float
Число → Rational	toRational	Точное представление текущего значения	toRational (0.1 :: Double) :: Rational
Rational → число	fromRational	Возможна потеря точности при переходе к Float/Double	fromRational (1 % 3) :: Double
Строка → число	read / reads	read может выбросить ошибку; безопаснее reads	read "42" :: Int
Число → строка	show	Универсально для всех типов	show (3.14 :: Double)

Создание своих типов данных

newtype

— обёртка над существующим типом
(эффективно и типобезопасно)

Подходит, когда нужно логически разделить «одинаковые» данные (например, ID и обычный Int, метры и килограммы), но без накладных расходов в рантайме.

Плюсы для практики:

чётко видно, что типы разные; компилятор не даст перепутать; в рантайме почти бесплатно.

```
newtype UserId = UserId Int
newtype DistanceM = DistanceM Double
```

```
uid :: UserId
uid = UserId 123
```

```
dist :: DistanceM
dist = DistanceM 5.5
```

```
unUserId :: UserId -> Int
unUserId (UserId x) = x
```

Создание своих типов данных

data – алгебраические типы данных (ADT)

Это основной способ создавать новые типы. Можно делать как «структуры» (один конструктор с полями), так и перечисления (несколько конструкторов).

1) «Структура» (как **struct** в других языках)

```
data Point = Point Double Double
```

```
origin :: Point
```

```
origin = Point 0 0
```

```
getX :: Point -> Double
```

```
getX (Point x _) = x
```

Создание своих типов данных

2) Перечисление (сумма типов)

```
data Color = Red | Green | Blue
```

```
data Shape = Circle Double      -- радиус  
           | Rectangle Double Double -- ширина, высота
```

Сопоставление с образцом (pattern matching):

```
area :: Shape -> Double  
area (Circle r)      = pi * r * r  
area (Rectangle w h) = w * h
```

Создание своих типов данных

Записи (record syntax) – удобные «структуры с именами полей»

```
data Person = Person
  { name    :: String
  , age     :: Int
  , height  :: Double
  }

alice :: Person
alice = Person { name = "Alice", age = 25, height = 1.7 }

-- Автоматически создаются селекторы: name, age, height
aliceAge :: Int
aliceAge = age alice
```

Можно обновлять поля через синтаксис записей:

```
aliceOlder :: Person
aliceOlder = alice { age = age alice + 1 }
```

Классы типов

Классы типов (type classes) — основной аналог интерфейса

Класс типа задаёт «контракт»: какие функции должны быть определены для типа, чтобы он принадлежал этому классу.

```
class Show a where
    show :: a -> String
```

```
class Eq a where
    (==) :: a -> a -> Bool
    (/=) :: a -> a -> Bool
```

Аналогия:

класс типа — это интерфейс;

instance — реализация интерфейса для конкретного типа.

Любой тип может стать экземпляром класса, если реализует требуемые функции:

```
data Color = Red | Green | Blue
```

```
instance Show Color where
    show Red    = "Red"
    show Green  = "Green"
    show Blue   = "Blue"
```

```
instance Eq Color where
    Red == Red    = True
    Green == Green = True
    Blue == Blue  = True
    _  == _       = False
```

deriving

deriving — это способ **автоматически сгенерировать реализации экземпляров классов типов** для твоего типа.

Это сильно экономит время и снижает риск ошибок, а для учебных задач — отличный способ сфокусироваться на логике, а не на шаблонном коде.

```
data Color = Red | Green | Blue
  deriving (Show, Eq)
```

Компилятор сам напишет instance Show Color и instance Eq Color.

Важные нюансы

Не всё можно вывести автоматически.

Например, Num, Monoid (если нет очевидной семантики), кастомные классы — их надо писать вручную.

Законы классов.

deriving Ord опирается на лексикографический порядок полей. Для некоторых типов это может быть не тем порядком, который ты хочешь (например, для температур, денег, приоритетов). Тогда лучше написать instance вручную.

Производительность.

Сгенерированный код обычно такой же эффективный, как и ручной.

Читаемость.

deriving делает определение типа компактным и понятным.

Bounded (GHC.Enum)

Bounded — это класс типов, который говорит, что у типа есть **минимальное и максимальное значение** (minBound и maxBound).

Чаще всего его используют вместе с Enum, чтобы удобно перебирать все возможные значения перечисления.

```
class Bounded a where
```

```
    minBound :: a
```

```
    maxBound :: a
```

```
data TaskStatus = New | InProgress | Done
```

```
    deriving (Show, Eq, Enum, Bounded)
```

```
allStatuses :: [TaskStatus]
```

```
allStatuses = [minBound .. maxBound]  -- [New, InProgress, Done]
```

Enum (GHC.Enum)

Enum — класс, который даёт операции перебора, инкремента, конвертации в/из целых чисел.

```
class Enum a where
  succ :: a -> a
  pred :: a -> a
  toEnum :: Int -> a
  fromEnum :: a -> Int
  enumFrom :: a -> [a]
  enumFromThen :: a -> a -> [a]
  enumFromTo :: a -> a -> [a]
  enumFromThenTo :: a -> a -> a -> [a]
```

Базовые функции класса Enum:

toEnum :: Int -> a — по номеру получить значение.

fromEnum :: a -> Int — по значению получить номер.

[Red .. Blue] — диапазон (работает благодаря Enum).

succ, **pred** — следующее/предыдущее значение.

```
data Color = Red | Green | Blue
  deriving (Show, Eq, Enum, Bounded)
```

```
fromEnum Red      -- 0
fromEnum Green    -- 1
fromEnum Blue     -- 2
```

```
toEnum 1 :: Color -- Green
```

```
[Red .. Blue]     -- [Red, Green, Blue]
```

```
succ Red          -- Green
pred Blue         -- Green
```

Enum (GHC.Enum)

Ручная реализация **Enum** (если нумерация не по порядку)

Иногда порядок конструкторов не совпадает с нужной нумерацией. Тогда пишут `instance` вручную.

Enum не гарантирует, что нумерация имеет смысл.

Для типов вроде `Shape` (круг, прямоугольник) `fromEnum` почти всегда бесполезен. Не выводите `Enum` «на всякий случай».

Диапазоны `[a .. b]` зависят от порядка конструкторов.

Если порядок изменится — логика диапазонов тоже изменится.

Не все типы должны быть **Enum**. Например, списки, функции — не имеют естественного порядка нумерации.

`succ/pred` могут выбросить ошибку, если выйти за границы.

```
data Priority = Low | Medium | High
```

```
instance Enum Priority where
```

```
    fromEnum Low      = 1
```

```
    fromEnum Medium   = 2
```

```
    fromEnum High     = 3
```

```
    toEnum 1 = Low
```

```
    toEnum 2 = Medium
```

```
    toEnum 3 = High
```

```
    toEnum _ = error "Invalid priority"
```

```
    succ Low      = Medium
```

```
    succ Medium   = High
```

```
    succ High     = error "No higher priority"
```

```
    pred High     = Medium
```

```
    pred Medium   = Low
```

```
    pred Low      = error "No lower priority"
```

Здесь нумерация начинается с 1, а не с 0, и поведение `succ/pred` контролируется явно.